

AI Assignment 1 final report

Group_A1_75

202309183

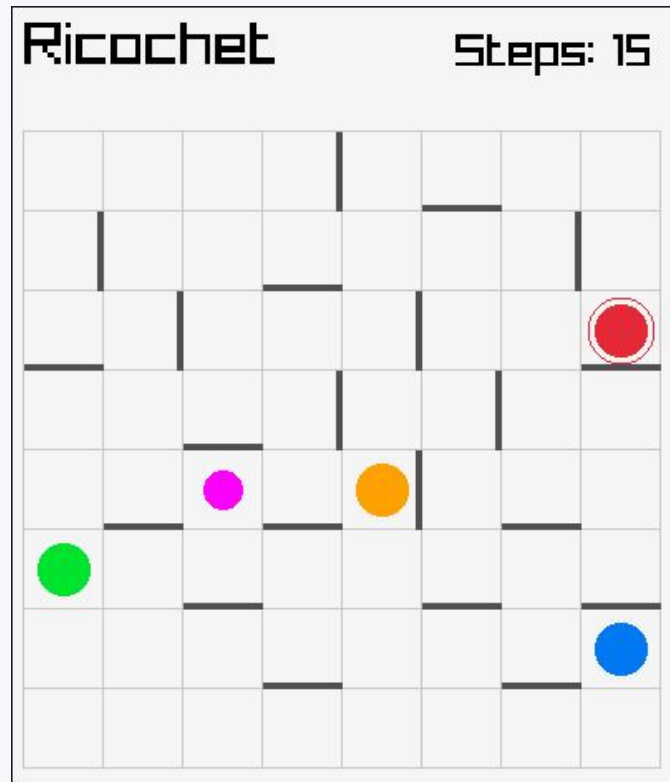
Mykyta Melnykovych

202305444

António Pedro Alves Pais

202207292

Rodrigo José Pires de Sousa



Ricochet Robots as a Heuristic Search Problem

One-player solitaire game

Player can move any of the robots can move in a straight line until they hit an obstacle (a wall or another robot)

Goal: reach target tile with any of the four robots

Objective: find the solution using the smallest amount of moves

References

Sources:

robotsevolved.com used as reference (no code was taken from it)

<https://www.riograndegames.com/wp-content/uploads/2013/02/RicochetRobots.pdf>

Search Problem Formulation

- Dynamic state: Represented by a struct with each robot's x and y position
- Static state: Represented by a grid that contains information about every tile (e.g. if there's a wall or if it's a goal tile). This part is not stored in search tree nodes, meaning it has almost no direct influence on the amount of memory used by the algorithms
- Initial state: Depends on the loaded level
- Objective test: Game is won if any of the robots is on the goal tile
- Operators: The player can choose a robot and move him in one direction
- Cost: Each move has a cost of 1
- Heuristic 1: Distance from the goal to the closest robot
- Heuristic 2: Sum of all robots' distances to the goal

Implementation

Language: C++

GUI: Raylib

Data structures: GameState, GameBoard, Node (contains a GameState, parent and children nodes, and other auxiliary data)

Algorithms we've implemented: BFS, DFS, IDS (limited at 10), Greedy (two versions with slightly different heuristics), A* and Weighted A* (with a weight of 1.2)

Features: GUI, movement, win detection, hint, loading custom levels from text files (format described in README), exporting solutions with execution metrics (time and memory used) to text files

Metrics - Solution steps

	BFS	DFS	IDS	GrV1	GrV2	A*	WA*
1.lvl	6	262	6	40	46	6	6
2.lvl	7	1953	7	10	19	7	7
3.lvl	15	6115	N/A	32	60	15	15
4.lvl	8	398	8	126	32	8	8
5.lvl	13	656	N/A	24	35	13	13
6.lvl	11	7361	N/A	112	29	11	11
7.lvl	16	N/A	N/A	43	35	16	16

Metrics - Execution time (ms)

	BFS	DFS	IDS	GrV1	GrV2	A*	WA*
1.lvl	1.68	2.12	13.66	4.11	11.05	0.96	1.06
2.lvl	31.03	19.93	2025.09	0.83	0.20	6.74	6.83
3.lvl	466.96	49.78	N/A	3.98	6.12	234.06	180.36
4.lvl	74.31	3.76	3371.9	47.02	3.21	2.61	3.07
5.lvl	179.09	5.00	N/A	0.65	0.51	5.96	5.50
6.lvl	1061.14	73.92	N/A	50.77	1.81	51.80	42.90
7.lvl	9202.03	N/A	N/A	9.00	5.24	1712.34	1908.42

Metrics - Memory usage (bytes)

	BFS	DFS	IDS	GrV1	GrV2	A*	WA*
1.lvl	53392	110920	672	216120	545272	43896	49344
2.lvl	1165080	1717056	968	65912	18808	377840	388480
3.lvl	13568144	3254584	N/A	233040	329152	6389688	5488960
4.lvl	2744816	316376	1064	2456072	200408	152528	179608
5.lvl	5612728	394088	N/A	42040	33400	268616	261272
6.lvl	31177248	5432264	N/A	2890608	133472	2444056	2116304
7.lvl	190803448	N/A	N/A	510344	363392	43605368	52113200

Conclusions

Uninformed algorithms (BFS, DFS, IDS) become impractical on larger levels, the state space increases exponentially and without heuristics, it is not possible to recognize “good” from “bad” states

Informed algorithms (Greedy, A*, Weighted A*) scale significantly better thanks to the use of heuristics that contribute a search towards to promising states

The quality of the heuristic is a critical factor: v2 (sum of Manhattan distances) outperforms v1(minimum) at more complex levels because it offers a better guidance since it takes in consideration all 4 players